

SHOPFLOW

Distributed Order Processing & Notification System

Powered by RabbitMQ

PRODUCT REQUIREMENTS DOCUMENT

Version	1.0.0
Team	Three Musketeers (Team 9)
Year	2026
Status	Approved for Implementation
Deployment	docker compose up --build

1. Executive Summary

ShopFlow is a fully Dockerized, simulated e-commerce backend that demonstrates production-grade distributed messaging using RabbitMQ. Every component - RabbitMQ cluster nodes, HAProxy load balancer, all producer and consumer services, the Chaos Control Panel, and the React dashboard - runs as a Docker Compose service. The entire system launches with a single command: `docker compose up --build`.

The system models a complete, realistic order lifecycle in which every component communicates exclusively via asynchronous messages routed through RabbitMQ, making the system resilient to individual service failures, traffic spikes, and broker node crashes.

Order Lifecycle Stages

- **Order Placement** - the customer places an order through the Producer API REST endpoint (port 8090). No manual CLI invocation is required.
- **Payment Processing** - payment worker pools (2 Docker containers) process the charge asynchronously via work queues.
- **Inventory Reservation** - stock is reserved without blocking the payment flow (2 Docker containers).
- **Notification Dispatch** - email, SMS, and push alerts fire simultaneously via a fanout exchange (3 Docker containers).
- **Audit Logging** - every event is captured and routed to the correct log sink via direct and topic exchanges.
- **Region Routing** - orders are forwarded to geography-specific processors via headers exchange routing.
- **Dead Letter Handling** - failed or expired messages are captured in a Dead Letter Exchange for forensic review. Entries are written to a shared Docker volume and exposed via the dashboard.
- **Fault Simulation** - a dedicated Chaos Control Panel (Docker container) lets operators stop, crash, and restart individual containers to demonstrate HA, DLX, and quorum queue failover live.

A real-time React dashboard (Docker container, port 3000) exposes the full internals of the messaging system - queue depths, message throughput, consumer status, exchange routing, cluster node health, and chaos controls - giving complete observability into the running system.

2. Problem Statement

2.1 The Synchronous Microservice Problem

In modern e-commerce platforms, services are naturally decomposed into independent units: Order Service, Payment Gateway, Inventory System, Notification Service, Logging Pipeline, and Region-specific Processors. When these services communicate synchronously (REST over HTTP), three critical failure modes emerge:

- **Cascading Failures** - if the Payment Gateway slows down, the Order Service thread pool fills up and the entire checkout flow stalls.

- **Tight Coupling** - every service must know the address and protocol of every other service it calls.
- **No Retry Semantics** - if a notification service crashes mid-send, the message is silently lost with no recovery path.
- **Request Blocking** - during traffic spikes, synchronous callers must wait for every downstream service to respond before continuing.

2.2 The Solution

Replace all inter-service calls with asynchronous messages through RabbitMQ. Each service publishes events to an exchange; downstream services subscribe to queues bound to that exchange. Services are fully decoupled - a publisher does not know or care who receives its messages. Key guarantees:

- If a consumer container crashes, RabbitMQ re-queues the unacknowledged message automatically on TCP disconnect.
- If a broker node fails, quorum queues elect a new leader and processing continues without data loss.
- Traffic spikes are absorbed by the queue - consumers process at their own rate without blocking producers.
- Every message path through the system is observable, auditable, and testable via the management UI and dashboard.

NOTE: The Chaos Control Panel (Section 9) is specifically designed to demonstrate these guarantees live - you can kill a consumer container, stop a broker node, or flood an exchange and watch the system recover in real time on the dashboard. All chaos actions are triggered via REST API and reflected instantly in the live UI.

3. Deployment Model - Fully Dockerized

Version 2.0.0 moves all components into Docker containers managed by a single docker-compose.yml. There are no local Python virtual environments, no manual process management, and no prerequisite installations beyond Docker Desktop.

3.1 Quick Start

The entire system - 24 containers - launches with one command:

```
bash shopflow_docker_setup.sh # generates all project files
cd shopflow
docker compose up --build      # builds images and starts all services
```

3.2 Service Startup Order

Docker Compose enforces a strict dependency chain via healthcheck conditions:

Stage	Services	Condition
1	rabbit1	Starts first; rabbit2 and rabbit3 wait for it to be healthy

Stage	Services	Condition
2	rabbit2, rabbit3	Start after rabbit1 is healthy (rabbitmq-diagnostics ping)
3	haproxy	Starts after rabbit1, rabbit2, rabbit3 are all healthy
4	cluster_init	One-shot container: declares all topology, then exits 0
5	All 14 consumers, producer_api, chaos_service	Start only after cluster_init completes successfully
6	frontend	Starts after chaos_service and producer_api are running

3.3 cluster_init - One-Shot Topology Bootstrapper

cluster_init is a special Docker Compose service that runs scripts/cluster_init.py, declares all exchanges, queues, and bindings in a single pass, then exits with code 0. It is configured with restart: "no" to prevent re-runs. All downstream services depend on it via service_completed_successfully, guaranteeing that the RabbitMQ topology is fully ready before any consumer or producer connects.

This is a critical architectural change from v1.x. Previously, every consumer declared its own topology at startup. Now, topology is declared exactly once by cluster_init, eliminating race conditions where consumers could start before their queues existed.

3.4 Docker Network & Static IPs

All services run inside the shopflow_net bridge network (subnet 172.20.0.0/24). Key static assignments:

Container	Static IP	Role
haproxy	172.20.0.10	Unified AMQP entrypoint for all producers/consumers
rabbit1	172.20.0.11	RabbitMQ node 1 (primary)
rabbit2	172.20.0.12	RabbitMQ node 2
rabbit3	172.20.0.13	RabbitMQ node 3
chaos_service	172.20.0.20	Chaos Control Panel API
frontend	172.20.0.30	React dashboard (nginx)
producer_api	172.20.0.40	Producer REST API (FastAPI)

3.5 Shared Log Volume

A shared_logs Docker named volume is mounted into all consumer containers, the DLX consumer, the Producer API, and the Chaos Service at /app/logs. This allows the Chaos Service to read DLX audit records and the dashboard to surface them in real time. Log files written to this volume:

- dead_letters.jsonl - written by dead_letter_consumer; read by chaos status endpoint
- error_logs.jsonl - written by log_error_consumer

- `notification_audit.jsonl` - written by `notif_audit_consumer`

3.6 Dockerfiles

The project uses two Dockerfiles:

- **Root Dockerfile (python:3.11-slim)** - shared base image used by `cluster_init`, all 14 consumers, and `producer_api`. Each docker-compose service uses the same built image and overrides the `command` field to select which Python module to run. This minimizes build time and image count.
- **chaos_service/Dockerfile (python:3.11-slim)** - separate image for the Chaos Control Panel because it has its own `requirements.txt` (includes docker SDK) and mounts `/var/run/docker.sock`.
- **frontend/Dockerfile (multi-stage)** - Node 20 Alpine builder stage runs `npm ci && npm run build`; output `dist/` is copied into `nginx:alpine` for serving. Port 80 inside container maps to host port 3000.

4. Use Case - ShopFlow E-Commerce Platform

4.1 Business Scenario

A customer places an order on ShopFlow. The following sequence of events must happen reliably, even if any individual container crashes mid-flight:

1. Customer submits order → Producer API (POST `/orders/publish`) publishes a message to all 5 exchange types simultaneously.
2. Payment Consumer (x2 containers) picks up the order from `payment_queue`, simulates a 2–5 second charge operation, and ACKs on success.
3. Inventory Consumer (x2 containers) picks up the same order from `inventory_queue`, reserves stock, and ACKs.
4. Email, SMS, and Push Consumers all receive the same order event from the fanout exchange - all three notifications fire in parallel.
5. The Topic Exchange routes `notification.email.*` to the email handler and `notification.sms.urgent` to the SMS handler - with priority-aware routing. The audit consumer receives everything via the `#` binding.
6. The Direct Exchange routes `error/warning` logs to `log_error_queue` (persisted to shared volume) and `info/debug` logs to `log_info_queue`.
7. The Headers Exchange routes the order to `eu_queue` (EU, JSON), `us_queue` (US, JSON), or `xml_legacy_queue` (XML format, any region).
8. If any consumer crashes mid-processing, the unACKed message is requeued automatically on TCP disconnect. After 3 failures, it is dead-lettered to `dead_letter_queue` and written to `dead_letters.jsonl` on the shared volume.

4.2 Service Map

Service / Container	Role	Exchange Pattern	Queue(s)
Order Producer (producer_api)	Publishes orders via REST API	All types	payment_queue, inventory_queue, + all exchanges
payment_consumer_1/2	Processes payment charges (2–5s)	Work Queue	payment_queue
inventory_consumer_1/2	Reserves stock (0.5–2s)	Work Queue	inventory_queue
email_consumer	Sends order confirmation emails	Fanout	email_queue
sms_consumer	Sends order SMS alerts	Fanout	sms_queue
push_consumer	Sends push notifications	Fanout	push_queue
log_error_consumer	Persists error/warning logs to shared volume	Direct	log_error_queue
log_info_consumer	Prints info/debug logs to container stdout	Direct	log_info_queue
notif_email_consumer	Routes email notifications by priority	Topic	notif_email_queue
notif_sms_consumer	Routes urgent SMS only	Topic	notif_sms_queue
notif_audit_consumer	Logs ALL notifications to audit file	Topic (#)	notif_audit_queue
eu_processor	GDPR-compliant EU order processing	Headers	eu_queue
us_processor	US tax calculation processing	Headers	us_queue
xml_legacy_consumer	XML legacy format processor	Headers	xml_legacy_queue
dead_letter_consumer	Audits all failed messages	DLX (direct)	dead_letter_queue

5. Functional Requirements

FR-01 - Work Queues (Load Balancing)

Work queues distribute time-consuming tasks across multiple worker containers. Producers push messages to a named queue; multiple consumers listen on the same queue and RabbitMQ assigns each message to exactly one consumer.

FR-01.1 Payment Processing Queue

- **Exchange:** Default (nameless) exchange - messages go directly to the named queue.
- **Queue:** payment_queue (durable: true, x-queue-type: quorum, x-message-ttl: 60000ms).
- **Producer:** Producer API publishes one message per order with payload: order_id, customer_name, customer_email, customer_phone, amount, currency, region, format, items[], timestamp.
- **Consumers:** payment_consumer_1 and payment_consumer_2 - two independent Docker containers running the same PaymentConsumer class.
- **Fair Dispatch:** prefetch_count=1 - a consumer receives the next message only after ACKing the previous one.
- **Simulated Work:** each consumer sleeps random(2.0, 5.0) seconds to simulate a payment gateway call.
- **Container startup:** both containers start only after cluster_init exits 0.

FR-01.2 Inventory Processing Queue

- **Queue:** inventory_queue (durable: true, x-queue-type: quorum, x-message-ttl: 60000ms).
- **Consumers:** inventory_consumer_1 and inventory_consumer_2 - two Docker containers.
- **Simulated Work:** random(0.5, 2.0) seconds to simulate a DB write for stock reservation.

FR-02 - Publisher-Subscriber: Fanout Exchange

- **Exchange:** order.events (type: fanout, durable: true).
- **Bound Queues:** email_queue, sms_queue, push_queue - all three receive every message.
- **Payload:** order_id, customer_name, customer_email, customer_phone, amount, currency, region, format, items[], timestamp.
- **email_consumer:** simulates sending a transactional email (0.2–0.8s).
- **sms_consumer:** simulates sending an SMS alert (0.1–0.5s).
- **push_consumer:** simulates sending a push notification (0.05–0.3s).
- **Guarantee:** ALL three consumers must receive every single order event. If one consumer container crashes, the other two continue unaffected.

FR-03 - Direct Exchange: Targeted Log Routing

- **Exchange:** logs.direct (type: direct, durable: true).
- **log_error_queue:** bound to routing keys 'error' AND 'warning'. Consumer writes JSON records to /app/logs/error_logs.jsonl on the shared volume.
- **log_info_queue:** bound to routing keys 'info' AND 'debug'. Consumer prints formatted lines to container stdout.
- **Both queues:** durable, quorum type, 60s TTL, DLX configured.

FR-04 - Topic Exchange: Notification Routing

- **Exchange:** notifications.topic (type: topic, durable: true).
- **Routing key schema:** notification.<channel>.<priority>

Binding patterns and destinations:

Routing Key Published	Consumer / Queue	Binding Pattern
notification.email.normal	notif_email_consumer	notification.email.*
notification.email.urgent	notif_email_consumer + notif_audit_consumer	notification.email.* / #
notification.sms.urgent	notif_sms_consumer + notif_audit_consumer	notification.sms.urgent / #
notification.sms.normal	notif_audit_consumer only	# (no SMS for normal priority)
notification.push.normal	notif_audit_consumer only	#

notif_audit_consumer writes all received notifications to /app/logs/notification_audit.jsonl (shared volume).

FR-05 - Headers Exchange: Region-Based Routing

- **Exchange:** orders.headers (type: headers, durable: true).

Bindings:

Container	Queue	Match Rule	Required Headers
eu_processor	eu_queue	x-match: all	region=EU, format=json
us_processor	us_queue	x-match: all	region=US, format=json
xml_legacy_consumer	xml_legacy_queue	x-match: any	format=xml (any region)

FR-06 - Message Persistence

- All queues declared with durable=True - survive broker restart.
- All producers publish with delivery_mode=2 (PERSISTENT). Messages written to Raft write-ahead log before confirming receipt.
- Quorum queues replicate the log across all 3 cluster nodes before acknowledging. A confirmed message resides on at least 2 of 3 nodes.
- Message TTL: 60,000ms (60 seconds) configured on all queues via x-message-ttl. Expired messages route to DLX automatically.

FR-07 - Manual Acknowledgments & Retry

All consumers use the BaseConsumer base class (_base_consumer.py) which implements:

- **ACK flow (success):** consumer processes message → calls channel.basic_ack(). RabbitMQ removes the message permanently.

- **NACK + requeue (failure, retries remaining):** `channel.basic_nack(requeue=True)`. RabbitMQ re-enqueues at the back of the queue.
- **NACK + dead-letter (exhausted retries):** after 3 failures (tracked via `x-death` header count), `channel.basic_nack(requeue=False)`. Message routes to DLX.
- **Auto-reconnect:** the `run()` loop catches `AMQPConnectionError` and retries connection every 5 seconds indefinitely. Consumer containers use `restart: on-failure` in `docker-compose`.
- **Consumer crash:** if a container is killed before ACKing, RabbitMQ detects TCP disconnect and requeues the unacked message automatically.

FR-08 - Dead Letter Exchange (DLX)

- **DLX Exchange:** `dead.letter.exchange` (type: `direct`, durable: `true`).
- **DLX Queue:** `dead_letter_queue` (durable, quorum type, no TTL - messages stay until manually reviewed).
- **DLX routing key:** `dlx.<source_queue_name>` (e.g. `dlx.payment_queue`, `dlx.email_queue`).

Messages are dead-lettered when:

- Message TTL expires (60,000ms) before a consumer processes it.
- Consumer NACKs with `requeue=False` after exhausting 3 retry attempts.
- **dead_letter_consumer container:** reads from `dead_letter_queue` continuously. Logs: `original_queue`, `routing_key`, `death_reason`, `received_at`, `retry_count`, full message body. Writes structured JSON to `/app/logs/dead_letters.jsonl` on the shared volume.
- **Dashboard exposure:** the Chaos Service reads `dead_letters.jsonl` and serves it via `GET /chaos/dlx/history` (up to 500 records). The DLX Audit tab on the frontend polls this endpoint.

FR-09 - Multi-Node RabbitMQ Cluster

- **Image:** `rabbitmq:3.13-management-alpine`.
- **Nodes:** `rabbit1` (172.20.0.11), `rabbit2` (172.20.0.12), `rabbit3` (172.20.0.13).
- **Cluster formation:** `classic_config` peer discovery. All nodes listed in `rabbitmq.conf` - no external registry required.
- **Erlang Cookie:** `SHOPFLOW_SECRET_COOKIE_2025` (shared secret enabling inter-node authentication).
- **Enabled plugins:** `rabbitmq_management`, `rabbitmq_peer_discovery_classic_config`, `rabbitmq_shovel`, `rabbitmq_shovel_management`.
- **HAProxy Load Balancer:** `haproxy:2.9-alpine`, port 5670 is the unified AMQP endpoint. Algorithm: `leastconn`. Health check: `tcp-check` every 5s, rise 2 / fall 3.
- **HAProxy Stats:** `http://localhost:8404/stats` (credentials: `admin / shopflow123`).
- **Virtual Host:** `shopflow` (all queues and exchanges live here).

- **RabbitMQ settings:** heartbeat=30s, vm_memory_high_watermark=0.6, disk_free_limit.relative=1.5.

FR-10 - Quorum Queues

All production queues are declared as quorum queues by cluster_init:

- **Declaration:** x-queue-type: quorum applied to all queues in declarations.py.
- **Replication:** minimum 3 replicas, one per cluster node. A write is confirmed only after a majority (2 of 3) acknowledge.
- **Leader election:** if the node holding the queue leader crashes, the remaining two nodes elect a new leader within seconds - no manual intervention required.
- **Additional args on all queues:** x-message-ttl: 60000, x-dead-letter-exchange: dead.letter.exchange, x-dead-letter-routing-key: dlx.<queue_name>.

FR-11 - Producer API (REST-Triggered Order Publishing)

The Producer API is a FastAPI service (producer_api container, port 8090) that replaces all manual command-line producer invocations. It wraps src/producers/order_producer.py and exposes HTTP endpoints the frontend and chaos scripts can call.

Endpoints:

Method + Path	Parameters	Description
POST /orders/publish	region, format, amount, currency	Publish one order through all 5 exchange types simultaneously.
POST /orders/batch	count, region, format	Publish N orders in sequence.
POST /orders/flood/{exchange}	exchange, count, routing_key	Flood a specific exchange with N messages (load test helper).
GET /health	-	Health probe returning {status: ok, service: producer_api}.

Swagger UI: <http://localhost:8090/docs>. The producer_api container shares the shared_logs volume.

FR-12 - Load Testing & Benchmarking

A Locust load test (tests/load/locustfile.py) simulates concurrent order placements via the Producer API (port 8090). Locust runs externally (pip install locust) or can be added as a Docker service.

Task distribution: 10 weight - POST /orders/publish (random region US/EU, random amount \$10–\$500); 3 weight - POST /orders/flood/order.events (5 messages); 1 weight - GET /health.

Test scenarios: Baseline (10 concurrent, 500 orders), Stress (ramp to 100 over 60s), Persistence cost (delivery_mode=1 vs 2), HA Failover (run baseline while stopping rabbit2 mid-test).

FR-13 - Real-Time Monitoring Dashboard

The React dashboard (frontend container, port 3000) is the primary interface for monitoring and controlling the ShopFlow system. Built with React 18 + Vite + Tailwind CSS, served via nginx inside Docker.

Tabs and panels:

Tab	Data Source	Content
Overview	RabbitMQ Management API /overview	Cluster health, message rates, total queued
Queues	RabbitMQ Management API /queues	Depth, ready, unacked, message rate per queue
Exchanges	RabbitMQ Management API /exchanges	Exchange type, bindings, publish rate
Consumers	RabbitMQ Management API /consumers	Active consumers, prefetch, queue, container
DLX Audit	GET /chaos/dlx/history	Dead letter records: queue, reason, retry count, body
Publisher	POST /orders/publish (producer_api)	Trigger single orders or batches from the UI
Chaos (⚡)	Chaos Control Panel API (port 8080)	All fault injection controls

Auto-refresh: all panels poll every 2 seconds. Frontend configuration via VITE_ env variables injected by docker-compose.yml.

API helpers: frontend/src/api/rabbitmq.js (RabbitMQ Management) and frontend/src/api/chaos.js (Chaos Panel + Producer API).

6. Non-Functional Requirements

Category	Requirement	Implementation
Portability	Zero host dependencies beyond Docker Desktop	Single docker compose up --build launches all 24 containers
Reproducibility	Identical environment on any machine	All config in docker-compose.yml + .env; pinned image versions
High Availability	No single point of failure in broker layer	3-node quorum cluster + HAProxy leastconn load balancing
Fault Tolerance	Message delivery despite consumer crash	Manual ACK + auto-requeue on TCP disconnect; max 3 retries then DLX
Observability	Full visibility into message flow	RabbitMQ Management UI (port 15672) + React Dashboard (port 3000)
Startup Order	Services start in correct dependency order	docker-compose healthcheck conditions + service_completed_successfully

Category	Requirement	Implementation
Data Durability	Messages survive broker restart	delivery_mode=2 + quorum queues with Raft WAL
Configurability	Runtime tunable without rebuild	All thresholds in .env: PREFETCH_COUNT, MAX_RETRIES, MESSAGE_TTL_MS
Teardown	Clean full reset	docker compose down -v removes all containers and volumes
Load Testing	Measurable throughput benchmarks	Locust against Producer API port 8090; results in tests/results/

7. System Architecture

7.1 Technology Stack

Layer	Technology	Version / Notes
Message Broker	RabbitMQ	3.13-management-alpine (quorum queues, Raft consensus)
AMQP Load Balancer	HAProxy	2.9-alpine, leastconn, port 5670
Consumer/Producer Runtime	Python	3.11-slim (shared Docker base image)
AMQP Client Library	pika	1.3.2 (BlockingConnection with heartbeat=30)
Producer REST API	FastAPI + uvicorn	0.110.0 / 0.29.0, port 8090, Swagger at /docs
Chaos Control Panel	FastAPI + Docker SDK	0.110.0 / docker==7.0.0, port 8080
HTTP Client (chaos)	httpx	0.27.0 (RabbitMQ Management API calls)
Frontend Framework	React 18 + Vite	18.2.0 / 5.2.0, Tailwind CSS 3.4.3
Frontend Charts	Recharts	2.12.3
Frontend Icons	lucide-react	0.363.0
Frontend Serving	nginx:alpine	Serves built Vite dist/, port 80 → host 3000
Container Orchestration	Docker Compose	v3.9, bridge network 172.20.0.0/24
Load Testing	Locust	External install, targets port 8090 Producer API

7.2 Port Map

Host Port	Container	Service
3000	frontend (nginx)	React Dashboard - http://localhost:3000
5670	haproxy	Unified AMQP endpoint (all producers/consumers connect here)
5672	rabbit1	Direct AMQP (debug only - bypass HAProxy)
5673	rabbit2	Direct AMQP (debug only)

Host Port	Container	Service
5674	rabbit3	Direct AMQP (debug only)
8080	chaos_service	Chaos Control Panel API + Swagger at /docs
8090	producer_api	Producer REST API + Swagger at /docs
8404	haproxy	HAProxy Stats UI - http://localhost:8404/stats
15672	rabbit1	RabbitMQ Management UI (admin/shopflow123)
15673	rabbit2	RabbitMQ Management UI (admin/shopflow123)
15674	rabbit3	RabbitMQ Management UI (admin/shopflow123)

7.3 Directory Structure

```

shopflow/
  Dockerfile                ← shared Python base image (cluster_init +
all consumers + producer_api)
  docker-compose.yml        ← all 24 services in one file
  .env / .env.example        ← runtime configuration
  requirements.txt           ← pika, fastapi, uvicorn, docker, httpx,
rich
  src/
    core/
      config.py              ← frozen Settings dataclass (reads .env)
      connection.py          ← get_connection() + get_channel() factory
      declarations.py        ← declare_all() - single source of truth
for all topology
      message_builder.py     ← build_properties(),
build_order_payload(), encode(), decode()
    consumers/
      _base_consumer.py      ← BaseConsumer: auto-reconnect, ACK/NACK,
retry, DLX
      payment_consumer.py    ← FR-01, 2-5s delay
      inventory_consumer.py  ← FR-01, 0.5-2s delay
      email/sms/push_consumer.py ← FR-02 fanout
      log_error/info_consumer.py ← FR-03 direct
      notif_email/sms/audit_consumer.py ← FR-04 topic
      eu/us_processor.py     ← FR-05 headers
      xml_legacy_consumer.py ← FR-05 headers
      dead_letter_consumer.py ← FR-08 DLX audit
    producers/
      order_producer.py      ← publish_order() - publishes to all 5
exchange types
    utils/
      retry.py               ← get_death_count(), should_dead_letter()

```

logger.py	← setup_logging() consistent across all
services	
producer_api/	
main.py	← FastAPI app with CORS
routes/order_routes.py	← /orders/publish, /orders/batch,
/orders/flood/{exchange}	
chaos_service/	
Dockerfile	← separate image (docker SDK + pika)
requirements.txt	
main.py	← FastAPI app, 4 route groups
routes/	
consumer_routes.py	←
/chaos/consumer/{stop,kill,pause,resume,start}	
broker_routes.py	← /chaos/broker/{stop,kill,start}
queue_routes.py	← /chaos/queue/{purge,poisson,flood},
/chaos/connections/drop-all	
status_routes.py	← /chaos/status, /chaos/dlx/history,
/chaos/restore-all	
services/	
docker_service.py	← Docker SDK wrapper (mounts
/var/run/docker.sock)	
rabbitmq_service.py	← RabbitMQ Management API + pika for queue
ops	
frontend/	
Dockerfile	← multi-stage: node:20-alpine builder →
nginx:alpine	
nginx.conf	← SPA routing (try_files), asset caching
src/	
api/rabbitmq.js	← fetchNodes, fetchOverview, fetchQueues,
etc.	
api/chaos.js	← all chaos + producer_api calls
hooks/useInterval.js	← auto-refresh hook
infrastructure/	
rabbitmq/rabbitmq.conf	← cluster formation, heartbeat, memory/disk
limits	
rabbitmq/definitions.json	← vhost, user, permissions
(exchanges/queues declared by cluster_init)	
rabbitmq/enabled_plugins	← management, peer_discovery, shovel
haproxy/haproxy.cfg	← amqp_frontend :5670, stats :8404,
leastconn backend	
scripts/	
cluster_init.py	← one-shot: declare_all() then sys.exit(0)
demo.sh	← automated 7-step chaos demo via curl
teardown.sh	← docker compose down -v + clean logs
tests/	

load/locustfile.py	← Locust tasks targeting producer_api port 8090
logs/ Docker volume)	← .gitkeep (populated at runtime via shared Docker volume)

8. Architecture Decisions Log

#	Decision	Rationale	Trade-off Accepted
1	Fully Dockerized - no local venv	Zero-dependency onboarding. Any machine with Docker runs the full system. Eliminates 'works on my machine' problems.	Slightly longer first build (image downloads). Mitigated by layer caching.
2	Single shared Dockerfile for all Python services	One build context; all consumers use the same image. Service selection via docker-compose command: override.	All consumers share the same Python deps. No per-service dep isolation (acceptable since deps are minimal).
3	cluster_init one-shot service	Centralises all RabbitMQ topology declarations. Eliminates race conditions. All consumers wait for service_completed_successfully.	If cluster_init fails, all dependent services refuse to start. Intentional - fail fast is correct behaviour.
4	HAProxy on port 5670 as AMQP endpoint	All Python services connect to haproxy:5670. Never directly to rabbit1/2/3. HAProxy redistributes load and handles node failures transparently.	One more hop per connection. Latency cost is negligible (<1ms on Docker bridge network).
5	Producer API (FastAPI) instead of CLI	The dashboard and chaos scripts need to trigger order publishing programmatically. A REST endpoint is far cleaner than exec-ing Python scripts inside containers.	Adds one container. Acceptable since it also serves as the Swagger demo interface.
6	Shared Docker volume for logs	dead_letter_consumer, log_error_consumer, and notif_audit_consumer write to shared_logs. The Chaos Service reads from it to serve the /dlx/history endpoint. Avoids needing a database.	Log files grow unboundedly. Acceptable for a demonstration system; add rotation in production.
7	restart: on-failure for consumers	Consumers auto-restart if they crash (e.g. unhandled exception). Combined with the BaseConsumer reconnect loop, this provides two layers of fault tolerance.	A consumer that crashes on every message will restart in a tight loop. Mitigated by DLX - poison messages are dead-lettered after 3 retries.
8	frontend served via nginx (not Vite dev server)	Production-grade serving. Vite dev server is for development only - not suitable inside Docker for demo reliability.	Changes to frontend source require docker compose build frontend. Use volume mounts + Vite dev server for active development.

#	Decision	Rationale	Trade-off Accepted
9	Quorum queues for all queues	Quorum queues replace deprecated classic mirrored queues. They use Raft consensus - writes confirmed by majority means no data loss on node failure.	Higher memory and disk usage per node vs classic queues. Accepted: reliability > resource efficiency for this system.
10	prefetch_count=1 for all consumers	Implements true fair dispatch. A slow consumer cannot accumulate a private backlog while fast consumers sit idle.	Reduces raw throughput vs higher prefetch. Correct for the demonstration where processing time variation is the key visual.

9. Chaos Control Panel - Fault Simulation Service

9.1 Purpose

The Chaos Control Panel is a dedicated Docker container (chaos_service) that provides on-demand fault injection for live demonstration of RabbitMQ's reliability guarantees. It is the most important component for understanding system behaviour under failure conditions.

Every chaos action triggers an immediate response visible in the live React dashboard - making the demo entirely self-explanatory without any manual intervention.

9.2 Architecture

The chaos_service container:

- Runs FastAPI on port 8080 with Swagger UI at /chaos docs.
- Mounts /var/run/docker.sock - gives it Docker SDK access to start, stop, pause, kill any container in the Compose stack by name.
- Calls the RabbitMQ Management HTTP API on all 3 nodes (tries each in order, fails over to next) via httpx to purge queues, drop connections, and read cluster state.
- Uses pika (connecting via HAProxy on port 5670) to inject synthetic poison messages into any queue.
- Reads /app/logs/dead_letters.jsonl from the shared volume to serve the DLX history endpoint.
- Knows the Producer API URL (http://172.20.0.40:8090) for advanced chaos scenarios that combine fault injection with order publishing.

9.3 Chaos Actions - Full Specification

API Endpoint	Action	What It Demonstrates
POST /chaos/consumer/stop	docker stop <container>	Queue depth rises; consumer reconnects; messages queued on heartbeat timeout
POST /chaos/consumer/kill	docker kill SIGKILL	Abrupt TCP disconnect; unACKed messages immediately queued by RabbitMQ
POST /chaos/consumer/pause	docker pause (cgroup freeze)	Consumer connection stays alive but processes nothing; messages accumulate in Unacked state

API Endpoint	Action	What It Demonstrates
POST /chaos/consumer/resume	docker unpause	Consumer resumes; Unacked messages immediately re-delivered
POST /chaos/consumer/start	docker start	Restart a stopped consumer; drains accumulated queue backlog
POST /chaos/broker/stop	docker stop rabbit{1,2,3}	Node goes offline; HAProxy reroutes traffic; quorum queue elects new leader
POST /chaos/broker/kill	docker kill rabbit{1,2,3}	Simulates sudden broker crash; HA failover speed visible in dashboard
POST /chaos/broker/start	docker start rabbit{1,2,3}	Node rejoins cluster; syncs Raft log; resumes processing
POST /chaos/queue/purge	DELETE /api/queues/.../contents	Clears all messages from a queue; demonstrates queue management
POST /chaos/queue/poison	pika publish invalid JSON	Injects messages that always fail processing; triggers DLX after 3 retries
POST /chaos/queue/flood	pika publish N messages	Overwhelms a specific exchange; demonstrates queue backlog and consumer catch-up
POST /chaos/connections/drop-all	DELETE all connections	Drops all AMQP connections; all consumers reconnect automatically via BaseConsumer loop
POST /chaos/restore-all	docker start all exited	Restores all stopped containers; system returns to steady state
GET /chaos/status	Docker SDK + RabbitMQ API	Returns all container statuses and cluster node health
GET /chaos/dlx/history?limit=N	Read dead_letters.jsonl	Returns last N dead letter records for DLX Audit tab (max 500)

9.4 Recommended Demo Script (scripts/demo.sh)

The automated demo script (bash scripts/demo.sh) executes all 7 chaos scenarios via curl against the live APIs. It can also be run manually tab-by-tab:

9. Publish 10 baseline orders: POST /orders/batch {count:10, region:"US"} → verify all queues receive messages.
10. Stop payment_consumer_1: POST /chaos/consumer/stop → publish 20 orders → watch payment_queue depth rise → restart consumer → observe instant drainage.
11. Kill email_consumer (SIGKILL): POST /chaos/consumer/kill → publish 10 orders → wait 60s for heartbeat timeout → verify messages requeued → restart consumer.
12. Inject 5 poison messages: POST /chaos/queue/poison {queue:"payment_queue", count:5} → watch DLX Audit panel increment after 3 retries each.
13. Stop rabbit2 (broker crash): POST /chaos/broker/stop {node:"rabbit2"} → publish 20 orders → observe HAProxy reroutes to rabbit1/rabbit3 → restart rabbit2.
14. Flood 500 messages: POST /chaos/queue/flood {exchange:"order.events", count:500} → observe queue spike and consumer catch-up throughput.

15. Restore all: POST /chaos/restore-all → system returns to steady state. All containers green on dashboard.

10. Performance vs. Reliability Trade-off Report

Configuration Decision	Reliability Gain	Performance Cost	ShopFlow Choice
delivery_mode=2 (persistent)	Messages survive broker restart	~20-30% throughput reduction	Always persistent - reliability preferred
Quorum queues (Raft)	Majority-acknowledged writes; zero loss on node failure	Higher memory + disk per node	All queues are quorum type
prefetch_count=1	True fair dispatch; no consumer backlog starvation	Lower raw throughput vs prefetch=10+	All consumers prefetch=1
Manual ACK mode	At-least-once delivery guaranteed	Extra network round-trip per message	All consumers use manual ACK
3-node cluster + HAProxy	Survives 1 node failure without downtime	Extra containers, leastconn overhead	Full HA cluster always on
TTL=60s on all queues	Expired messages auto-routed to DLX - no queue bloat	Legitimate slow consumers may lose messages	60s default; tunable via env
Max retries = 3	Poison messages dead-lettered after 3 attempts	3x processing overhead for failing messages	3 retries; tunable via MAX_RETRIES env

11. Deliverables Checklist

	Deliverable	Location / Notes
✓	shopflow_docker_setup.sh	Single script that generates the entire project structure
✓	docker-compose.yml	24 services, bridge network, healthcheck dependency chain
✓	Root Dockerfile (Python 3.11-slim)	Shared image for cluster_init + all 14 consumers + producer_api
✓	chaos_service/Dockerfile	Separate image with Docker SDK + pika + httpx
✓	frontend/Dockerfile (multi-stage)	Node 20 builder → nginx:alpine serving on port 3000
✓	infrastructure/rabbitmq/rabbitmq.conf	3-node classic_config peer discovery, heartbeat, memory/disk limits
✓	infrastructure/rabbitmq/definitions.json	vhost, user, permissions (topology via cluster_init)
✓	infrastructure/haproxy/haproxy.cfg	leastconn AMQP frontend :5670, stats :8404
✓	src/core/config.py	Frozen Settings dataclass, env-var driven
✓	src/core/connection.py	get_connection() with retry logic via HAProxy
✓	src/core/declarations.py	declare_all() - single topology source of truth

	Deliverable	Location / Notes
✓	src/core/message_builder.py	build_properties(), build_order_payload(), encode(), decode()
✓	src/consumers/_base_consumer.py	BaseConsumer: auto-reconnect, ACK/NACK, retry, DLX routing
✓	14 consumer modules	FR-01 through FR-08 (payment x2, inventory x2, email, sms, push, log_error, log_info, notif_email, notif_sms, notif_audit, eu, us, xml_legacy, dead_letter)
✓	src/producers/order_producer.py	publish_order() - all 5 exchange types in one call
✓	producer_api/main.py + routes	FastAPI service, port 8090, /orders/publish, /orders/batch, /orders/flood
✓	scripts/cluster_init.py	One-shot topology init, exits 0 on success
✓	chaos_service/ (full)	main.py + 4 route files + docker_service.py + rabbitmq_service.py
✓	frontend/src/ (full)	App.jsx (7 tabs), api/rabbitmq.js, api/chaos.js, hooks/useInterval.js
✓	tests/load/locustfile.py	Locust tasks targeting Producer API port 8090
✓	scripts/demo.sh	Automated 7-step chaos demo via curl
✓	scripts/teardown.sh	docker compose down -v + clean logs
✓	.env + .env.example	All runtime config: credentials, ports, consumer settings
✓	README.md	Quick start, port map, architecture diagram, demo instructions

12. Glossary

Term	Definition
AMQP	Advanced Message Queuing Protocol - the wire protocol used by RabbitMQ and all producers/consumers.
BaseConsumer	Python base class (_base_consumer.py) providing auto-reconnect, manual ACK/NACK, retry counting, and DLX routing for all 14 consumer containers.
cluster_init	One-shot Docker Compose service that declares all RabbitMQ topology (exchanges, queues, bindings) and exits 0. All downstream services depend on it via service_completed_successfully.
Dead Letter Exchange (DLX)	dead.letter.exchange - a direct exchange that receives messages when they expire (TTL), are NACKed with requeue=False, or exhaust retries.
Docker SDK (docker-py)	Python library used by chaos_service to manage container lifecycle (start/stop/pause/kill) via the /var/run/docker.sock socket.

Term	Definition
Exchange	A RabbitMQ routing component. Producers publish to exchanges; exchanges route messages to queues based on type and bindings (fanout, direct, topic, headers, default).
Fair Dispatch	prefetch_count=1 - each consumer receives the next message only after ACKing the previous one. Prevents backlog starvation.
HAProxy	haproxy:2.9-alpine load balancer providing a unified AMQP endpoint on port 5670 using leastconn balancing across all 3 RabbitMQ nodes.
Manual ACK	Consumers explicitly call basic_ack() or basic_nack() to confirm or reject each message. Guarantees at-least-once delivery.
Producer API	FastAPI service (producer_api container, port 8090) that wraps order_producer.py and exposes REST endpoints for triggering order publishing without CLI access.
Quorum Queue	RabbitMQ queue type using Raft consensus for replication. Writes confirmed by majority of nodes - zero message loss on single node failure.
Raft	Consensus algorithm used by quorum queues. A write is confirmed only after a majority (2 of 3 nodes) acknowledge it.
shared_logs	Docker named volume mounted into consumers, producer_api, and chaos_service at /app/logs. Enables the Chaos Service to serve DLX history without a database.
shopflow_net	Docker bridge network (172.20.0.0/24) connecting all containers with static IP assignments.
TTL (Message TTL)	x-message-ttl: 60000ms. Messages not consumed within 60 seconds are automatically dead-lettered.
Virtual Host (vhost)	shopflow - all exchanges, queues, and bindings in this project live in this isolated namespace.
x-death header	RabbitMQ header added to dead-lettered messages. Contains death reason, original queue, timestamp, and retry count. Consumers inspect it to implement max retry logic.